

PULSE PROJECT DOCUMENTATION

A practical guide to the personalized content dashboard

Prepared by Chethan Nazre S | SDE Intern Frontend Development Assignment

Repository: <https://github.com/ChethanNazre/Pulse>

Live application: <https://mypulse.vercel.app/>

PROJECT OVERVIEW

Pulse is a responsive web dashboard that brings news, movie recommendations, and public social posts into one place. Users can choose interests, search across sources, browse trending items, save favorites, and arrange or dismiss feed cards. Settings also include appearance, language, and a browser-local demo profile.

This guide explains both what a user can do and how the main parts of the application work together. It describes the implementation in the Pulse repository and calls out service credentials and test-environment limits so the setup can be repeated without assuming that external services are already configured.

AT A GLANCE

Area	Pulse implementation
Application	Next.js 14 App Router with React 18 and TypeScript
Shared state	Redux Toolkit slices and asynchronous thunks
Content sources	NewsAPI and TMDB server routes; Mastodon.social public tag timelines
User controls	Categories, search, favorites, trending, reorder, feedback, theme, language
Persistence	Selected preferences and user interactions stored in the browser
Verification	Jest and React Testing Library suite; Playwright suite is present, with a documented Windows runner limitation

1. PURPOSE AND OBJECTIVES

The assignment calls for a personalized content experience rather than a collection of unrelated API demos. Pulse addresses that by using one dashboard shell and a shared content-card model while keeping provider-specific fetching behind dedicated routes.

- Let users choose topics and adjust their feed without creating a server account.
- Combine news, film recommendations, and public social posts in a single browsing experience.
- Make loading, empty, partial-failure, and retry states understandable.
- Support search, trending, favorites, card feedback, and reordering through accessible controls.
- Keep provider secrets on the server and make local setup reproducible through documented environment variables.

2. TECHNOLOGY STACK

Technology	Role in Pulse
Next.js 14.2 / React 18.3	App Router pages, shared layout, server route handlers, and interactive UI.
TypeScript 6	Types for components, shared content records, store state, and API handling.
Redux Toolkit 2.13 / React Redux 9.3	Shared feature state, reducers, selectors, and async thunks.
Framer Motion 14	Card interactions, drag reordering, and short interface transitions.
i18next / react-i18next	English and Spanish for the main dashboard interface.
Jest / React Testing Library	Unit and integration tests for state, utilities, components, and feed behavior.
Playwright	Browser end-to-end scenarios; execution currently has a Windows spawn limitation in the documented environment.
CSS and Next.js assets	Responsive layout, theme tokens, focus states, reduced-motion rules, and lazy-loaded imagery.

3. ARCHITECTURE AND PROJECT STRUCTURE

Pulse uses a small layered structure. App Router pages select the main view, shared components provide the shell and cards, Redux holds cross-page state, and API routes isolate provider access. This keeps UI actions separate from external service credentials and response formats.

Location	Responsibility
src/app	Dashboard, Trending, Favorites, Settings, legal pages, and API route handlers.
src/components	Application shell, header, sidebar, feed, content cards, dialogs, loading skeletons, and shared UI.
src/store	Preferences, profile, feed, trending, favorites, and card-feedback slices.
src/lib	Browser persistence, client/server API helpers, sample data, site configuration, and translations.
src/hooks	Reusable UI behavior, including debounced search input.
__tests__ and e2e	Jest/RTL tests and Playwright browser scenarios.

The server routes are same-origin endpoints: `/api/news`, `/api/movies`, `/api/social`, and `/api/events`. A shared content type lets the UI render common fields such as source, title, summary, image, date, and destination while retaining provider-specific metadata where needed.

4. MAIN FEATURES

Personalized feed and preferences

The Feed view combines available items from news, movie discovery, and social sources. The Settings page exposes six topic categories, appearance, language, profile controls, and feed-interaction controls. Category selections are held in the preferences slice and influence the requests or filtering used by the feed and Trending views.

The user flow is deliberately immediate: choose interests, return to the feed, and browse the resulting sources. The site can only show live NewsAPI and TMDB content when those provider credentials are configured in the hosting or local environment. Missing or unavailable providers are reported as errors rather than being replaced with invented items.

Content cards and interactions

Cards adapt their visible metadata to the source. News may include publisher imagery and publication dates; movie items use TMDB artwork and movie details; social items show public Mastodon post content. When an image is absent, Pulse uses a neutral fallback rather than implying a real photograph.

- Favorite an item and revisit it from the Favorites view.
- Dismiss a card or give recommendation feedback to reduce similar suggestions; Settings can restore items or reset those choices.
- Reorder cards by dragging the handle, or use the labeled move-up and move-down buttons with a keyboard.
- Use the news call to action to continue to the publisher; the demo profile gate is a browser-only UI flow.

Trending, favorites, and pagination

Trending has its own page and loads ranked content through the same source-oriented data flow. Favorites is a separate view backed by the favorites slice. The feed requests content in pages: an `IntersectionObserver` can request the next page as the reader approaches the end, and a visible Load more button remains available when automatic loading is unavailable. The thunk guards against duplicate page requests.

Responsive layout, theme, language, and motion

On larger screens, the dashboard uses a sidebar and a sticky header. On narrow screens, navigation moves to a compact bottom bar and the content reflows. The header provides search and navigation controls. Light and dark themes use CSS custom properties; the initial appearance can follow the system theme before a saved choice is applied. English and Spanish are available for the main interface, while legal pages and some source text remain in their original language.

Framer Motion supports short card and reorder interactions. Reduced-motion preferences are respected. Loading skeletons and status messages help explain when a view is waiting for data or cannot retrieve it.

5. API INTEGRATION AND ASYNCHRONOUS BEHAVIOR

Source	Request path and behavior	Configuration
News	The server route calls NewsAPI and maps returned articles into the common feed model. Query values are bounded and category input is allow-listed.	NEWS_API_KEY
Movies	The server route calls TMDB discovery/search and maps movie results for card rendering.	TMDB_API_KEY (Read Access Token)
Social	The server route reads public tag timelines from Mastodon.social; it does not require a user token. Sample posts are opt-in.	No key; USE_SAMPLE_DATA is optional and false by default.
Refresh events	The SSE route emits a refresh signal periodically; the client refetches current sources. It is not an upstream provider push stream.	No provider key.

News and movie credentials are read only by server-side handlers. Source requests can run concurrently, and one provider's failure can be reported as a warning while successful sources still render. Async state tracks request progress, and request-race protection prevents an older search response from overwriting a newer one. Sample content is available only when explicitly enabled and is intended for a local test or demo source.

6. REDUX TOOLKIT AND DATA FLOW

Redux Toolkit is the shared state layer for interactions that appear across pages. The store is configured in `src/store/index.ts` and exposed to the React tree through the provider component. Typed hooks keep component access consistent.

Slice	What it stores
preferences	Selected categories, theme, language, and feed-related options.
feed	Feed items, query and pagination state, loading/error status, and request coordination.
trending	Trending records and their asynchronous load state.
favorites	The user's saved content items.
feedback	Per-card dismissals and recommendation feedback.
auth/profile	A local demo profile used by the sign-in dialog and news-link flow; it is not server authentication.

Typical feed request

- A user changes a category, enters a search, or requests another page.
- The component dispatches a typed Redux action or async thunk with the current query and preferences.
- The client API helper requests the same-origin Next.js route; the route calls its provider and maps the response.
- The thunk records loading, success, or failure and updates the relevant slice. Independent source failures can be surfaced without discarding successful results.

- Selectors provide the current items and state to the feed and card components.
- A persistence helper restores selected user choices after mount and writes supported changes to browser storage.

7. SEARCH AND DEBOUNCE

Search is available across news, movies, and social content. The input is debounced for 400 milliseconds: Pulse waits until typing pauses before passing the settled query into Redux and the API request flow. This avoids issuing a provider request for every keystroke and makes the search experience calmer on slower connections.

The debounce behavior is implemented in `src/hooks/useDebounce.ts` and covered by a Jest test. Request coordination in the feed state protects the interface from stale results when the user quickly changes the query.

8. DRAG-AND-DROP AND KEYBOARD ORGANIZATION

Framer Motion reordering lets the user drag a card to a new position. The card also exposes labeled up and down controls, which provide a keyboard-friendly alternative to dragging. The feed slice owns the visible order during the session. The README identifies browser persistence for selected preferences, favorites, profile, and card feedback; it does not claim that card order itself survives a refresh.

9. PERSISTENCE AND USER PROFILE

Pulse stores selected settings and interactions in the current browser. Hydration is performed after the app mounts so default state does not overwrite saved choices. Persisted values include preferences, favorites, the mock profile, and per-card feedback. The profile is browser-local and contains a display name and email; it has no password, server account, cross-device sync, or authorization role.

Browser storage is user-controlled and is not suitable for secrets. Pulse does not store provider keys in `localStorage`. The current documentation does not claim server-synced preferences or persistent card ordering.

10. TESTING AND VERIFICATION

The project includes Jest and React Testing Library coverage for reducers and slices, utilities, content cards, sign-in gating, translations, server API behavior, event refresh behavior, and feed loading, empty, error, and retry flows. The suite is available through the test script in `package.json`.

A Playwright end-to-end suite is also present for key flows such as search, ordering, favorites, settings, Trending, pagination, recovery, profile/language, responsive cards, and legal pages. In the documented Windows sandbox, the runner exits with `spawn EPERM` before test collection. This is an environment limitation, not a passing end-to-end result; run the suite on a standard local terminal or CI runner before reporting it as verified.

Useful local checks are `npm run typecheck`, `npm run lint`, `npm test -- --runInBand`, `npm run test:e2e`, and `npm run build`. Only report the outcome after running each command in the environment where the submission is prepared.

11. SECURITY AND PRIVACY

- Keep NEWS_API_KEY and TMDB_API_KEY in server-side environment variables. Never add them to a NEXT_PUBLIC_ variable or commit them to source control.
- The News and movie route handlers bound or validate request parameters before contacting providers.
- External links use safe browser behavior such as noopener and norereferrer.
- Treat values from localStorage as user-controlled input; browser storage is not an authentication or authorization boundary.
- The demo profile only gates a browser interaction. It does not secure news or personal data on the server.
- Review the privacy and terms pages for the real site operator, contact address, hosting behavior, and any analytics before submission.

12. SETUP AND RUN

Requirements: Node.js 20 or newer and npm. Clone the repository, install dependencies, configure the provider and public-site values, then start the Next.js development server.

- 1.git clone <https://github.com/ChethanNazre/Pulse.git>
- 2.cd Pulse
- 3.npm install
- 4.Copy .env.example to .env.local and fill in the required values.
- 5.npm run dev
- 6.Open <http://localhost:3000> in a browser.

ENVIRONMENT VARIABLES

Variable	Purpose
NEWS_API_KEY	NewsAPI credential read by /api/news.
TMDB_API_KEY	TMDB API Read Access Token read by /api/movies.
USE_SAMPLE_DATA	Keep false for live content; true opts into clearly marked sample data for a local test/demo.
NEXT_PUBLIC_SITE_URL	Canonical public site URL, including https://.
NEXT_PUBLIC_SITE_OPERATOR	Operator name shown on legal pages.
NEXT_PUBLIC_CONTACT_EMAIL	Contact address displayed for privacy/legal requests.

Create the NewsAPI and TMDB credentials through their provider accounts, then store them in .env.local or the host's environment settings. Restart the development server after changing local environment values. Do not commit .env.local. The social route uses Mastodon.social's public API and does not require a key.

PRODUCTION AND TEST COMMANDS

Purpose	Command
Type check	npm run typecheck
Lint	npm run lint
Unit and integration tests	npm test -- --runInBand
Browser end-to-end tests	npm run test:e2e
Production build	npm run build
Start production server	npm start

13. TYPICAL USER JOURNEY

- Open Pulse and scan the Feed for news, movie recommendations, and social posts.
- Use Settings to choose topics and appearance or language preferences.
- Search for a topic and wait for the 400 ms debounce to settle before results refresh.
- Open Trending to browse ranked items or save a card to Favorites.
- Reorder a card by dragging or by using its keyboard move controls; dismiss or tune recommendations when useful.
- Return later in the same browser to see the preferences and interactions that Pulse persists locally.

14. CURRENT LIMITATIONS

- NewsAPI and TMDB content depends on valid provider credentials and each provider's availability, plan, and rate limits.
- Mastodon.social posts depend on public tag activity and instance access; coverage may be uneven or rate-limited.
- The SSE connection signals a refresh every 30 seconds; it does not receive live content pushes from the content providers.
- The sign-in/profile feature is a browser-local demonstration, not a real identity system.
- English and Spanish cover the primary interface; legal copy and some source content remain in their original language.
- The Playwright suite requires a standard local or CI environment to work around the documented Windows spawn EPERM limitation.
- A formal WCAG audit, screen-reader review, measured production performance profile, and end-to-end pass report are not included in the current evidence.

15. FUTURE ENHANCEMENTS

- Replace the mock profile with a supported identity provider and server-side account preferences.
- Add provider-aware caching, retry/backoff, freshness labels, and monitoring for rate limits and outages.
- Improve recommendation relevance using transparent, user-controlled feedback signals.
- Complete a formal accessibility review and run browser tests in CI across representative viewport sizes.

- Review provider terms and attribution, then consider additional sources only where access is permitted.
- Measure real-user performance and consider list virtualization if feed sizes grow substantially.

CONCLUSION

Pulse demonstrates a practical personalized dashboard built with Next.js, React, TypeScript, and Redux Toolkit. Its main user flows connect preferences, multi-source content, search, trending, favorites, card feedback, and responsive navigation. Provider access stays behind server routes, while selected settings and interactions remain local to the user's browser.

Before submitting, configure the live NewsAPI and TMDB credentials in the deployment environment, run the available checks, verify the legal-page details, and record the demo against the deployed site. Report test results only after those checks have completed.